

Auxiliary and Modal Stacking in Thai: An XLE Implementation

Piyapath Thomas Spencer Srisomyos
Matr-Nr: 01/1599718
piyapath.spencer@uni-konstanz.de

August 21, 2026

1 Rationale and Background

Consider the following Thai sentence:

- (1) เขา คง อาจ จะ ควร ต้อง กำลัง จะ ได้
k^hǎw k^hoŋ ʔà:t tɕàʔ k^hu:an tɔ̃:ŋ kam.laŋ tɕàʔ dâj
3SG MOOD_{eval} MOD_{epi} MOOD_{irr} MOD_{alet.wk} MOD_{alet.str} ASP_{prog} ASP_{prosp} MOOD_{real}
ทำ ได้ แล้ว ละ มั้ง
t^ham dâj lé:w láʔ máŋ
do ASP_{abil} ASP_{pftv} MOOD_{sfp} MOOD_{hedge}
‘Perhaps (s)he is probably supposed to be able to do it by now.’

Fourteen words, one main verb (ทำ, ‘do’; in bold), and thirteen function words surrounding it, each contributing a different piece of grammatical features, in a *fixed* linear order. As Thai verbs do not inflect, it marks tense, aspect, mood, and modality almost entirely with preverbal (and a few postverbal) particles rather than with verbal morphology. When more than one such particle is present, they appear in a strict linear order, traditionally described (c.f. [Iwasaki and Ingkaphirom 2005](#); [Spencer forthcoming](#)).

This report documents a grammar fragment that parses sentences like (1) and correctly rejects any ungrammatical reorderings. Thus, the grammar is deliberately small, comprising the closed-class auxiliary/modal inventory plus a minimal clause structure (one subject, one optional object, one main verb); a small, provisional serial-verb/object-control addition (§ 3.6) tests that same stacking against a more complex verb phrase, confirming it has no effect. Otherwise, no tokenisation, semantics, or disambiguation of ambiguous readings is attempted. Its purpose is to show that the *fixed linear order* and the *differing syntactic behaviour* of these function words can be captured with a small, principled rule system, rather than an ad hoc list of allowed orderings.

The pattern in Table 1 is not a Thai idiosyncrasy. [Cinque \(1999\)](#) observes, cross-linguistically, that preverbal adverbs and auxiliaries appear in a rigid relative order across unrelated languages. However, Cinque’s account is *cartographic*¹, and an unfilled projection is then not absent from the

¹Universal grammar provides one syntactic projection per interpretable mood/modal/tense/aspect distinction, merged in one universally fixed order, in *every* clause of *every* language, whether or not that projection’s head or specifier is filled by overt material in a given sentence.

Position	Word	Tranlation	Grammatical category	
1	คง	‘probably’ (inference)	evaluative mood	MOOD _{EVAL}
2	อาจ	‘might’ (possibility)	epistemic modal	MOD _{EPIS}
3	เคย	‘used to’ (experiential)	past tense	TNS _{PST}
4	จะ	‘would’ (irrealis)	irrealis mood	MOOD _{IRR}
5	เคย	(anterior, heterosemous with 3)	anterior tense	TNS _{ANT}
6	ควร	‘should’ (weak necessity)	weak alethic modal	MOD _{ALET.WK}
7	ต้อง	‘must’ (strong necessity), heterosemous with 9	strong alethic modal	MOD _{ALET.STR}
8	กำลัง	‘is ...-ing’	progressive aspect	ASP _{PROG}
9	ต้อง	‘has to’ (obligation)	deontic modal	MOD _{OBL}
10	จะ	‘about to’, heterosemous with 4	prospective aspect	ASP _{PROSP}
11	ได้	‘already/got to’, heterosemous with 12–13	realis aspect	ASP _{REAL}
V+ (OBJ)				
12	ได้	‘can’ (ability)	completive/abilitative aspect	ASP _{COMPL/ABIL}
13	แล้ว	‘already’ (perfective)	perfective aspect	ASP _{PFTV}
14	ละ	sentence-final particle	speech-act particle	MOOD _{SFP}
15	มั้ง	hedging particle (‘maybe’)	hedge particle	MOOD _{HEDGE}

Table 1: The fixed linear hierarchy modeled by the grammar, outermost (leftmost) to innermost (rightmost/lowest). Four words are *heterosemous*: the same phonological form occupies two different positions with two different grammatical behaviors.

tree but present with a null head. On such view, a bare Thai sentence like เขาทำ (‘s/he does (it)’) would not have a two-word verbal spine; it would contain, silently stacked above the verb, one empty functional projection for every mood, modal, tense, and aspect distinction in the universal hierarchy this sentence happens not to spell out i.e. fifteen or more null heads, immediately dominating one another, for a two-word sentence.

Thai raises a direct question whether capturing the restriction *require* positing that much silent structure? This report aims:

1. to document this phenomenon of Thai itself and
2. more generally, to show that an XLE implementation *can* be built in line with what cartographic syntax predicts, despite its economy principle.

§ 2 takes up the question from the LFG formalism that the grammar is actually written in.²

2 Grammar Architecture

The sixteen positions in Table 1 do not all behave the same way syntactically, even though they all *look* like preverbal particles on the surface. The grammar follows a distinction standard in LFG grammar-writing practice (the ParGram Grammar Writer’s Cookbook; Butt et al. 1999, §3.5–3.6) between two kinds of function word.

²For Setup and installation, see Appendix A.

Auxiliaries are fully grammaticalised markers with no predicate-argument structure of their own. Progressive กำลัง (1) does not have a subject or a complement in any meaningful sense, but simply *marks a feature* (“this event is ongoing”) on the very same clause the main verb heads, attaching with $\uparrow = \downarrow$ (or $\wedge = !$) being a co-head, contributing one feature to the f-structure it shares with everything else at its level.

Modals are raising predicates. ควร ‘should’ (1) behaves syntactically like English *should/ought to* in that it has its own predicate (‘ควร-should<SUBJ,XCOMP>’), takes the rest of the clause as an XCOMP complement, and shares its subject with that complement via functional control, attaching with $(\uparrow\text{XCOMP}) = \downarrow$.

Six of the sixteen positions are modals in this sense (evaluative คง, epistemic อาจ, irrealis จะ, weak and strong alethic ควร/ต้อง, deontic ต้อง), and the rest are auxiliaries. Since there is no inflection in Thai, on the surface, distinction is invisible on the surface but they actually is different. As § 3 illustrates, a sentence with two modals present has an f-structure with two levels of XCOMP nesting, whilst a sentence with only auxiliaries present has one flat f-structure no matter how many auxiliaries are stacked. That is, the fixed *linear* order of Table 1 holds regardless of this split. In other words, the order is concerning *c-structure* and the modal/auxiliary split is concerning *f-structure*.

2.1 From a Cinque grammar to LFG grammar

Ideally, the grammar should match the cartographic commitments (§ 1), as well as satisfy LFG’s economy of expression. Figure 1 shows a minimalist/cartographic derivation of เขาควรทำ ‘He should do (something)’, in the style of Cinque (1999). On this tree, each pre-verbal projection reflects per row of Table 1, with a phonologically null head except $\text{ModP}_{\text{alet}(\text{wk})}$, where ควร merges directly. เขา raises from SpecvP Position to SpecTP_{pst}, and ทำ raises from V^0 to v^0 via trace/copy.

To transfer this idea to LFG grammar, the first thing to do is to encode the whole hierarchy of Thai auxiliaries directly, i.e. one phrase-structure category per position, combined by recursive descent. Of course, LFG would not like that. To be precise, a modal’s subject-sharing is a functional-control equation (i.e. $(\uparrow\text{SUBJ}) = (\uparrow\text{XCOMP SUBJ})$) in its lexical entry, so there is no trace to build and nowhere for a moved subject to sit; meanwhile, the verb is simply base-generated where it is pronounced, so there is no v^0 landing site and no $v\text{P}/\text{VP}$ split to project. What does carry over is the ordering itself: fifteen positions in a fixed sequence, any subset of which may be filled. A clause may have one word between subject and verb, or ten, and their relative order is the same either way.

In the pre-final version of this grammar, the rule is a single parametrised category, $\text{AuxP}[_{\text{pos}} \dots]$; each alternative applies only when $_{\text{pos}}$ equals one particular value, as illustrated below:

```
AuxP[_pos $ {pred modepi moodirr modaletweak modaletstrong modobl}] -->
{...
|Aux[Mod, alet, weak]:  $\wedge = !$ 
  @SUBJ-SUBJ-CONTROL;
Aux[modaletstrong]: ( $\wedge$  XCOMP)=!
  _pos = modaletweak
|Aux[modaletstrong]:  $\wedge = !$ 
  _pos = modaletweak
...}.
```

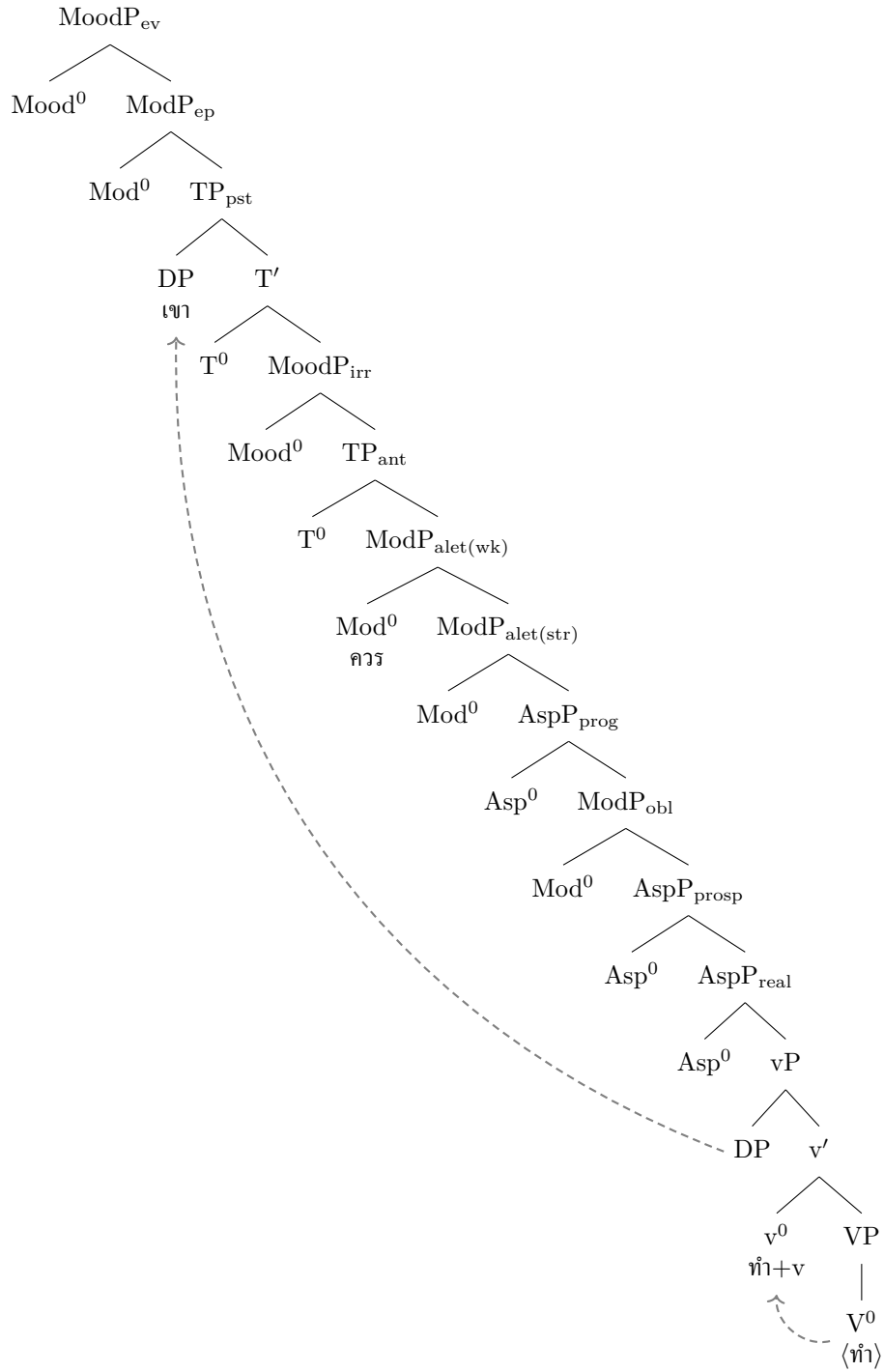


Figure 1: A minimalist/cartographic derivation of เขาควรทำ, in the style of Cinque (1999)

The first alternative consumes `ควร` ‘should’, shares the subject with the clause below it, and passes what remains to the next position down; the line `_pos = modaletweak` is the guard, and it is what makes this alternative the weak alethic position rather than any other. The second alternative is the same step with nothing consumed, for clauses in which `ควร` is absent. To illustrate, a node can only use the `modaletweak` alternative when its own `_pos` is exactly `modaletweak`, which implies that a distinct node for every position above it has already been built and passed through.

With this, as in [Figure 2](#), a c-structure node is generated for every position on every parse, whether or not that position’s word is present. For `เขาควรทำ` ‘(s)he should do (it)’, this produces `AuxP[pred]`, `AuxP[modepi]`, `AuxP[moodirr]`, `AuxP[modaletweak]` (host for `ควร`), `AuxP[modaletstrong]`, `AuxP[modobl]`, and then `VP` (host for `ทำ` ‘do’), at seven nodes below `S`, four of which are empty.

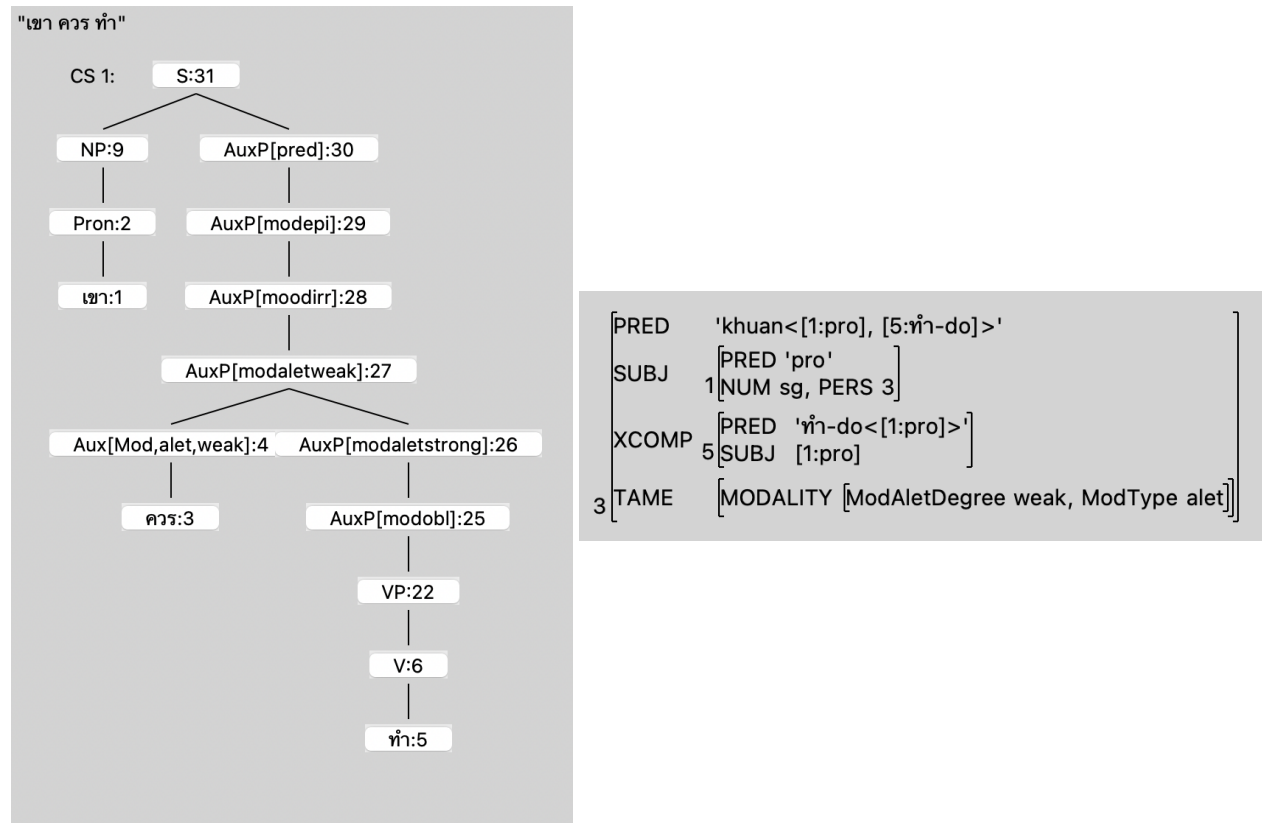


Figure 2: The pre-final c-structure for `เขาควรทำ` (left) and its f-structure (right). The f-structure is already the right one; it is the tree on the left that the rest of this section is about.

It should be confident to say that this grammar is formally correct LFG, as every equation is a proper f-structure equation and nothing is stipulated by tree geometry. It was also fully restrictive, in the sense that sequential recursive descent through a fixed chain rejects every reordering, and the archived copy of it matches the test suite exactly. However, it does not yet satisfy LFG’s economy principle, because its c-structure is still built on cartography’s logic: those four empty nodes are in the tree because the hierarchy says a position of that kind exists, which is structurally the same commitment § 1 attributes to cartography, even though nothing was pronounced there.

To fix this, the instruction is reversed, so that a node is built when a word licenses it and not otherwise. Every row of [Table 1](#) still keeps a category of its own (`MoodEval`, `ModEpi`, `TensePast`, and so on down to `Pftv`, `SpeechAct` and `Hedge`), so no position is given up; the difference is that a

category is only ever built when its own word is in the string. For this to be possible, each preverbal rule has to name every position that may legally follow it, instead of naming only the next one:

```
|Aux[Mod, alet, weak]: ^=!  
  @SUBJ-SUBJ-CONTROL;  
{VP[ModAletStrong]: (^ XCOMP)=!  
  _pos = ModAletWeak  
|VP[AspProg]: (^ XCOMP)=!  
  _pos = ModAletWeak  
...  
|VP[Pftv]: (^ XCOMP)=!  
  _pos = ModAletWeak  
|VP: (^ XCOMP)=!  
  _pos = ModAletWeak}
```

That list covers everything below `ควร` in the hierarchy and stops there. The positions above it are absent from the list, and that absence is what keeps the order fixed, since once `ควร` has been consumed there is no route back up to a position that should have preceded it. Because every legal destination is offered directly, `ควร` attaches to whichever position is filled next, and the unfilled ones in between are never built at all. For the same sentence, this collapses the seven nodes down to two, as in [Figure 3](#), and neither of them is empty.

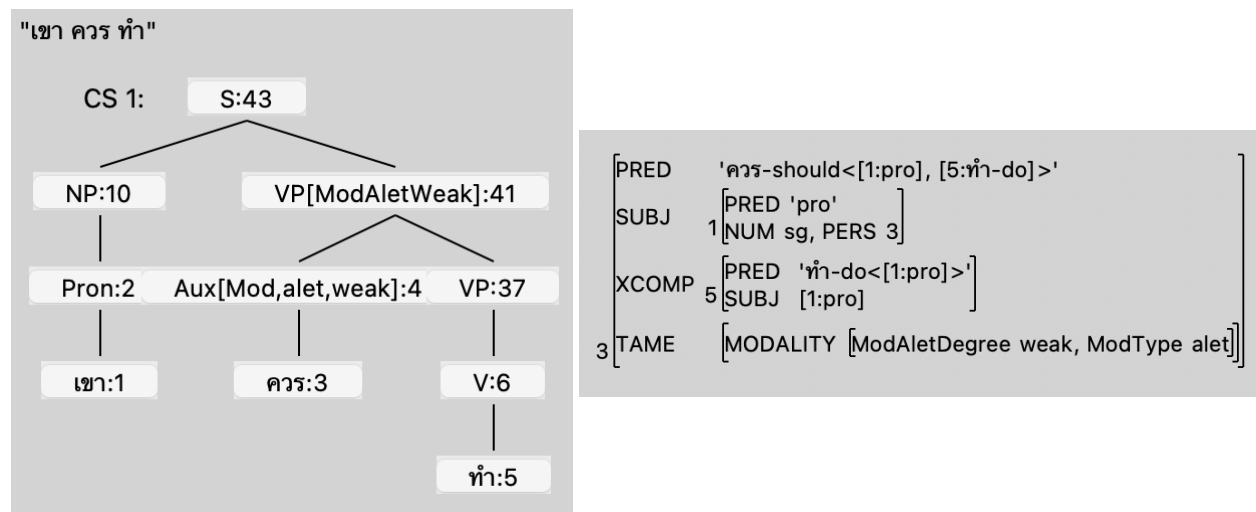


Figure 3: The current c-structure and f-structure for `เขาควรทำ`, to be compared with the left-hand tree of [Figure 2](#). You can see that the f-structure is unchanged.

Eleven of the fifteen positions are preverbal and work in this way. The remaining four cannot, because they have nothing after them to name: `ได้` ‘can’ and `แล้ว` ‘already’ come after the verb, while `ละ` and `นี่` are clause-final particles, the latter roughly a spoken hedge. Each of these takes what precedes it as its argument instead, and consumes its own word last:

```
{VP[SpeechAct]: ^=!  
|VP[MoodEval]: ^=!  
...  
|VP: ^=!}  
Aux[Mood, hedge]: ^=!  
  _pos = Hedge
```

This is the same shape as the preverbal rules, i.e. a functional head with its complement, only written in the other direction, because that is where Thai puts these particular words.

One category, VP, is not a position in the hierarchy at all, and it stays for a separate reason: it is where a verb and its object sit together. An object is attached with (NP: @OBJ), a template which puts the object on its mother’s f-structure, so a verb and its object have to be siblings under a node that carries the verb’s own f-structure. If the verb were attached directly to the modal above it, the object would land in the modal’s f-structure instead, one level too shallow for what a transitive verb such as กิน ‘eat’ subcategorises for, and the parse then fails: เขาคควรกินข้าว ‘(s)he should eat rice’ gets no analysis at all, while เขาคควรทำ, whose verb takes no object, is unaffected. VP is also what lets object control nest, since บอก ‘tell’ takes a VP complement which may itself contain another.

Nevertheless, the current design of grammar has a cost of its own. Since each of the four final particles is now a category in its own right, stacking two of them costs two nodes, whereas the pre-final grammar’s flat verb node held both as optional daughters; เขาทำได้แล้ว ‘(s)he can already do it’ comes out as VP[Pftv] over VP[Comp1] over the verb. Counted node by node, then, neither design wins everywhere. What the current one gains is that the status of a position is always visible in the label of a node, and never has to be inferred from an optional daughter inside a neighbouring rule, and that no node is built for a word which is not there. All 45 sentences of the test suite (§ 4) parse with the counts they are supposed to have.

thai-rules.lfg implements this design, and the worked examples of § 3 were regenerated against it, so every tree and every AVM shown there is real output from the current grammar.

2.2 Economy of expression

According to Bresnan et al. (2016, 90), “[a]ll syntactic phrase structure nodes are optional and are not used unless required by independent principles (completeness, coherence, semantic expressivity)” (Bresnan et al., 2016, 90) That is, a c-structure node earns its place by being independently needed to support something actually pronounced or actually required by an f-structure well-formedness condition, never inserted merely because a universal template says a position of that kind should exist there. XLE formally *supports* null terminal nodes and they were used this way in early LFG for long-distance dependencies (Kaplan and Bresnan, 1982), but since Kaplan and Zaenen (1989) replaced that with *functional uncertainty*, using empty categories for phenomena functional uncertainty already covers has been unnecessary, and hence not standard practice, for over three decades.

The pre-final grammar and the current one are equivalent on restrictiveness in that both generate every attested combination and reject every reordering. What changed is purely what the grammar *posits* consistent with LFG’s premise that the c- to f-structure correspondence is language-specific (Belyaev, 2023, 4). In other words, the fixed order is a fact *this grammar* states about Thai, by saying which positions each position may be followed by, not one inherited from a universal template that every language must then be shown to obey.

2.3 When do siblings share an attribute?

All auxiliary/modal features are nested under one complex attribute, TAME (Tense-Aspect-Mood-modality), itself organised into four branches (TENSE, ASPECT, MOOD, MODALITY) so that, for in-

stance, tense values and aspect values can never accidentally collide. Within a branch, a two-value attribute like `MoodType` (`eval|irr`) is correct precisely when the two values can never co-occur on the same local f-structure; when co-occurrence is real, as with the two clause-final particles `ล่ะ` and `มั้ง` (เขาทำแล้วล่ะมั้ง, ‘s/he’s done it, you know, maybe’), they remain two independent attributes (`MoodSfp`, `MoodHedge`) instead.

Making the grammar usable on real (unsegmented) Thai script requires an external word-segmentation step wired into XLE’s runtime configuration, since Thai marks phrase boundaries with spaces but not word boundaries; that mechanism is implementation logistics rather than part of the linguistic argument, so it is described in full, alongside installation, in Appendix A.

3 Worked Examples

This section demonstrates six sentences through the grammar in full: two simple cases establishing the auxiliary/modal contrast directly (3.1, 3.4), one confirming that contrast scales to two stacked modals at once (3.2), one demonstrating heterosemy resolution by position (3.3), one ungrammatical string demonstrating how the fixed order is enforced as a structural fact rather than a stipulated list (3.5), and, last, one glimpse of the grammar’s separate, provisional serial-verb-construction extension (3.6) for the sake of completeness. Five of these six are part of the 45-sentence regression suite and glossed again in the Appendix; 3.4 is retained here to illustrate flat auxiliary co-heading with zero embedding, but was dropped from the regression suite itself after its status could not be confirmed with confidence.

3.1 A single modal

- (2) เขา ควร ทำ
 k^hǎw k^hu:an t^ham
 3SG MOD_{ALET.WK} do
 “(S)he should do it.”

ควร is a modal (§ 2) and has its own predicate and takes the rest of the clause as its complement. This is visible directly in both structures (Figure 3): the c-structure has an embedding, and the f-structure has a `XCOMP`, whose `SUBJ` is not locally filled but identified with the matrix `SUBJ` by the functional control equation stated in `ควร`’s lexical entry. Also, no c-structure node appears for any of the positions which sit between `ควร` and the verb in the abstract hierarchy of Table 1, i.e. `ModAletStrong`, `AspProg`, `ModObl`, `AspProsp` and `AspReal`. None of their words is present, so none of them is built; `VP[ModAletWeak]` names `VP` directly among its continuations (§ 2.1). `VP` is the node which carries the verb together with any object it takes, and here it is the only node below `ควร`.

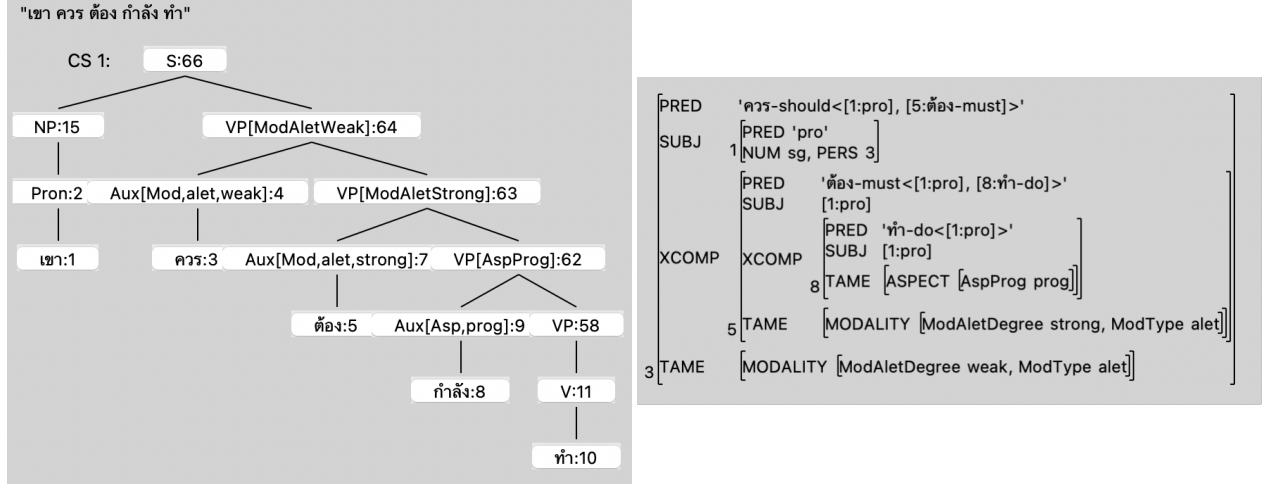


Figure 4: เขาควรต้องกำลังทำ: two modals, two XCOMP layers, one shared SUBJ (index 1). Each modal opens one XCOMP, so the two-modal claim is visible directly in the depth of the f-structure. กำลัง is an auxiliary rather than a modal, so it opens nothing; it heads a c-structure node of its own and contributes PROG to the f-structure of the clause below ต้อง, which is the clause the verb also heads.

3.2 Two modals stacked

3.3 Heterosemy resolved by position

- (3) เขา เคย จะ เคย ทำ
 k^hǎw k^hɤːj tɕàʔ k^hɤːj t^ham
 3SG TNS_{PST} MOOD_{IRR} TNS_{ANT} do

‘(S)he’d apparently done it before.’ syntactically well-formed; included as a test case for heterosemy resolution, not offered as an idiomatic utterance

This sentence uses เคย *twice*, in its two different heterosemous readings, with the one true modal of the sentence (จะ, irrealis) sandwiched between them. Figure 5 shows why this works. The first เคย (position 3, past tense) attaches *flatly* ($\sim =!$), so VP[TensePast] and VP[MoodIrr] are *two distinct c-structure nodes but one f-structure*, connected by structure-sharing rather than embedding, and PAST ends up on the same f-structure as จะ’s own IRR. จะ, by contrast, is a modal: it opens a genuine new XCOMP layer. The second เคย (position 5, anterior tense) attaches flatly again, but inside *that* embedded layer, so ANT lands on the embedded f-structure, the one the verb heads. The result is an f-structure with exactly one embedding, for จะ, and the two heterosemous เคย tokens sit on either side of it. Both attach as co-heads rather than as raising predicates, which is what lets each of them add a tense feature to a clause without adding depth to it. This example is also the one that originally caught the first of the two wrong turns described in § 2.1: that attempt at eliminating vacuous nodes let the “flat auxiliary present, modal absent” case fall through the cracks, since neither Mood,eval nor Mod,epi is present in this sentence, only the auxiliary is.

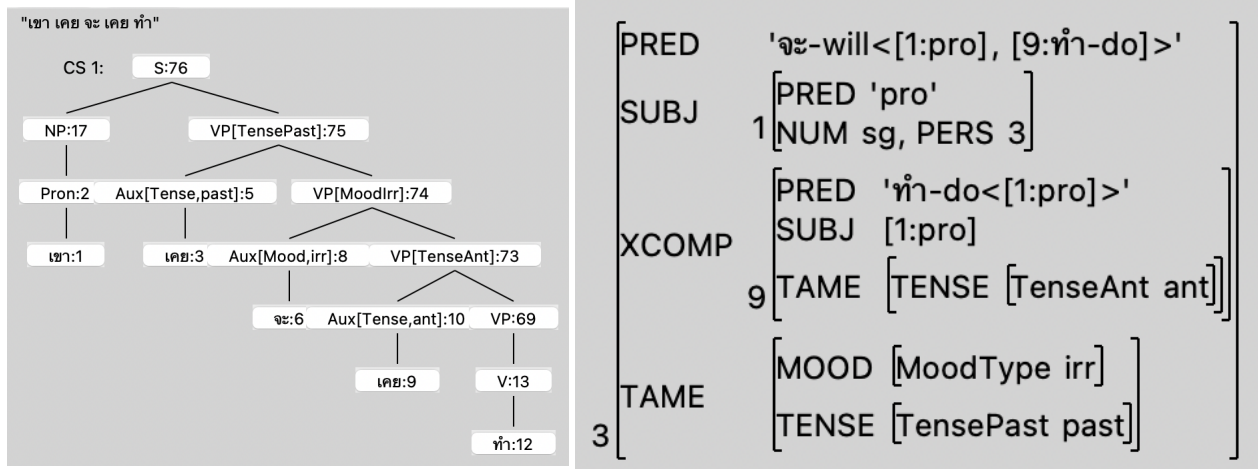


Figure 5: เขาเคยจะเคยทำ: two auxiliary attachments surrounding one real modal embedding. VP[TensePast] and VP[MoodIrr] are different tree nodes, both annotated $\uparrow=\downarrow$, which map to the *same* f-structure, so PAST and IRR appear together on it. จะ then opens the one XCOMP of the sentence, and the second เคย contributes ANT to that embedded f-structure, which the verb also heads.

3.4 Auxiliary stacking

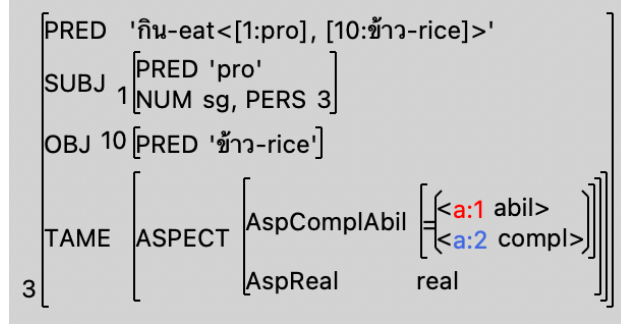
3

- (4) เขา ได้ กิน ข้าว ได้
 k^hǎw dâj kin k^hâ:w dâj
 3SG MOOD_{REAL} eat rice ASP_{ABIL}
 ‘(S)he got to eat rice, and can do it again.’

A useful contrast to both previous examples: this sentence contains *no* modal at all, only two auxiliary attachments of ได้, once in its preverbal Asp,real reading and once in its postverbal Asp,abil reading, which is the same heterosemy strategy as § 3.3, but with zero raising anywhere in the sentence. Every word ends up on *one flat f-structure*:

No XCOMP appears anywhere, regardless of how many auxiliary positions are filled, suggesting that stacking *auxiliaries* (as opposed to *modals*) never adds f-structure depth, only additional flat features on a single f-structure. This is the direct structural correlate of the auxiliary/modal distinction argued for in § 2.

³เขาได้กินข้าวได้ was removed from `testsuite.lfg` after review raised doubt about whether other speakers of Thai could actually accept ได้ stacked pre- and post-verbally without further material to support it such semantic/pragmatic judgment this syntax-only fragment cannot make either way. It is kept here only because the structural point above is independent of that judgment; it is no longer glossed in the Appendix or counted among the 45 regression-suite sentences.



3.5 An ungrammatical order

- (5) *เขา กำลัง ควร ทำ
 k^hǎw kam.laŋ k^hu:an t^ham
 3SG ASP_{PROG} MOD_{ALET.WK} do
 intended: ‘(s)he’s currently supposed to do it’

กำลัง (progressive, position 8) and ควร (weak alethic modal, position 6) appear here in the *wrong* relative order, and the grammar rejects this string because กำลัง commits the derivation to VP[AspProg], and ModAletWeak is not among the continuations that VP[AspProg] names (§ 2.1). Its list runs from ModOb1 downwards, i.e. over the positions below กำลัง in the hierarchy and no others, so once กำลัง has been consumed there is no route back up to license ควร, and the third word of the string cannot attach to *any* category. The fixed linear order in Table 1 is therefore not stipulated anywhere as a separate constraint; rather, it follows from which continuations each position names.

3.6 A provisional sketch for serial verb constructions

- (6) เขา เดิน ออก ไป กิน ข้าว
 k^hǎw dɔ:n ʔò:k paj kin k^hâ:w
 3SG walk exit go eat rice
 ‘(S)he walks out to eat rice.’

The five examples above are all about the auxiliary/modal system that is this report’s subject. The grammar also includes a small, provisional extension for Thai *serial verb constructions* e.g. เดิน...ออก...ไป...กิน, one clause, four verbs, one subject. Not a claim that Thai SVCs are theoretically settled as they remain genuinely unsettled in LFG themselves, and § 3.6 lays out the competing analyses and why this one was chosen, but only that this much of the mechanism is real, tested, and in this particular grammar.

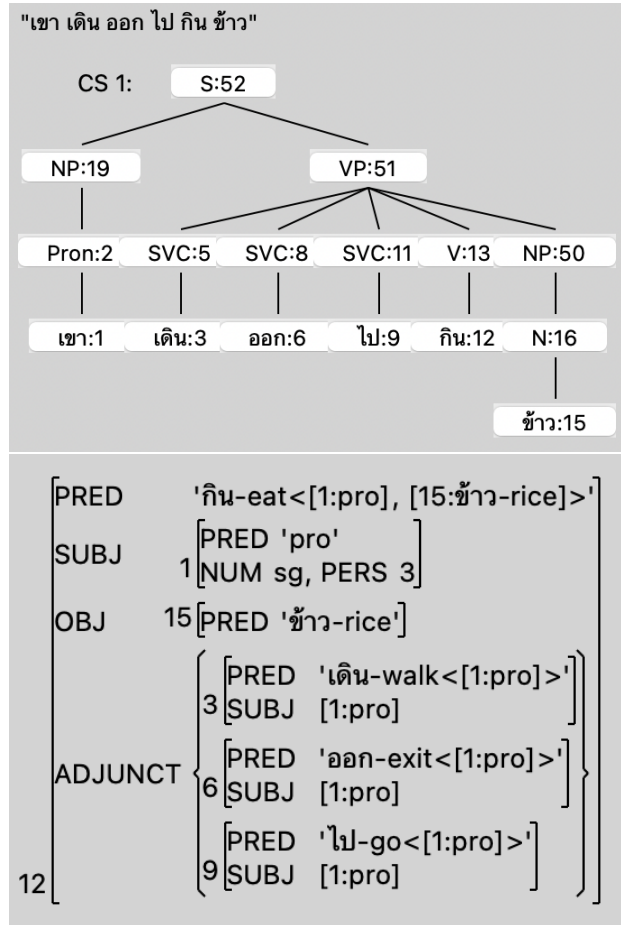


Figure 6: เขาเดินออกไปกินข้าว: a flat, four-verb serial chain — no XCOMP anywhere, three ADJUNCT-set members sharing one SUBJ with the head verb กิน. Verified against real XLE output; testsuite.lfg #5.

4 Alternative?

There is a shorter route to the same solution counts, and it is worth mentioning why the grammar does not take it. XLE implements ranking in the style of Optimality Theory (Crouch et al., 2011): a rule can add a mark with `Mark $ o::*`, the `OPTIMALITYORDER` field of the configuration ranks the marks, and only the best-ranked analyses are reported. The pre-final grammar could therefore have been left exactly as it was, with a mark added to each step that passes through an unfilled position, so that derivations building vacuous nodes rank below derivations that do not. This was implemented and run, and every sentence of the test suite then reports the expected count.

What it does not do is stop the vacuous nodes from being built. XLE reports such a parse as `1+1 solutions`: one optimal analysis and one *unoptimal* one, both fully constructed and both well-formed. `เขาคกรทำ` comes out as `1+1`, `เขาคกรทำ` as `1+1`, `เขาคกรกินข้าวได้` as `2+2`. The derivation with an empty node for every unfilled position is still licensed by the grammar and still a complete analysis of the string; the mark records that it is unwanted without withdrawing it. Economy of expression is a statement about which nodes a grammar builds, and it cannot be satisfied by a filter applied to nodes already built.

Another objection is that the mechanism is too powerful to be informative. A mark may be attached to any alternative of any rule, and the ranking is a free parameter of the configuration, so for any set of unwanted analyses there is some assignment of marks that suppresses exactly those and leaves the rest. A device that can be made to fit any outcome predicts none as it could not have turned out to be wrong about Thai, or about anything else. What it yields is a permissive grammar plus a list of the cases to be discounted, and a list of exceptions is the data restated rather than derived. The current grammar contains no such list. Both the fixed order and the absence of vacuous nodes follow from one statement the rules need anyway which positions each position may be followed by so ungrammaticality and economy are consequences of the same thing rather than two separate stipulations. Ranking would trade that result for a description of it.

None of this counts against Optimality Theory itself. In OT proper the constraints are universal and independently motivated, and only their ranking varies by language (Prince and Smolensky, 2004); that is what gives a ranking its explanatory force, and the same holds of the declarative OT-LFG tradition built on it (Kuhn, 2003). A mark invented for one grammar in order to cancel a redundancy that the same grammar introduced has none of that standing, which is a diacritic and not a constraint. XLE's marks earn their place where the phenomenon really is preference e.g. disambiguation, robust parsing of ill-formed input, choosing among competing outputs in generation. Thai's ordering restriction is not of that kind. `*เขาคกำลังกรทำ` is not a dispreferred way of saying anything; it has no analysis at all (§ 3.5), which is what the grammar says about it without any ranking machinery. The configuration is therefore left at `OPTIMALITYORDER NOGOOD`, with no marks declared anywhere in the rules.

5 Conclusion and Discussion

This report's purpose, I repeated here again, is to document the Thai data particularly with a fixed, cross-categorical ordering restriction and, for higher level generalisation, to show that an LFG implementation *can* be built what cartographic syntax à la Cinque (1999) predicts for it, regardless of having a different theoretical motivation. Built with no cartographic intent, the pre-final grammar (§ 2.1) nonetheless ends up committed to cartography, one node per position

regardless of content. That such a grammar is reachable at all, inside a formalism that does not presuppose it, is this project’s central finding. The two architectures are, moreover, observationally equivalent throughout and neither Table 1’s ordering facts nor the rejection of reorderings like 3.5 distinguish them. What they are not equivalent on is parsimony i.e. economy of expression accounts for the same facts with a c-structure that scales with what is pronounced, where a cartographic one does not.

It is important to note that the scope of this project is syntactic. The postverbal ได้ complete/ability ambiguity and ต้อง’s strong-alethic/deontic heterosemy are left genuinely unresolved, the lexicon is minimal, and future work includes wider coverage, a semantic/discourse component for the open ambiguities, and testing the same technique against other rigidly-ordered languages. Furthermore, the grammar is additionally tested against Thai *serial verb constructions* (several verbs, one subject, no coordinator), which is to some extent theoretically unsettled in LFG, to confirm auxiliary/modal stacking has no effect on a more complex verb phrase. A minimal, provisional extension (flat ADJUNCT-sharing, in `thai-rules.lfg`) parses serial chains, chains under a modal/auxiliary, and object control with a droppable addressee, with no regressions elsewhere, and does not claim the theoretical question is resolved.

References

- Belyaev, Oleg. 2023. Core concepts of LFG. In Mary Dalrymple (ed.), *Handbook of lexical functional grammar*, 23–96. Berlin: Language Science Press.
- Bresnan, Joan, Ash Asudeh, Ida Toivonen and Stephen Wechsler. 2016. *Lexical-Functional Syntax* (Blackwell Textbooks in Linguistics 16). Malden, MA: Wiley-Blackwell 2nd edn.
- Butt, Miriam, Tracy Holloway King, María-Eugenia Niño and Frédérique Segond. 1999. *A grammar writer’s cookbook*. Stanford: CSLI Publications.
- Cinque, Guglielmo. 1999. *Adverbs and functional heads: A cross-linguistic perspective*. New York: Oxford University Press.
- Crouch, Dick, Mary Dalrymple, Ronald M. Kaplan, Tracy Holloway King, John T. Maxwell, III and Paula Newman. 2011. XLE documentation. Tech. rep. Palo Alto Research Center.
- Iwasaki, Shoichi and Preeya Ingkaphirom. 2005. *A reference grammar of Thai*. Cambridge: Cambridge University Press.
- Kaplan, Ronald M. and Joan Bresnan. 1982. Lexical-Functional Grammar: A formal system for grammatical representation. In Joan Bresnan (ed.), *The mental representation of grammatical relations*, 173–281. Cambridge, MA: The MIT Press.
- Kaplan, Ronald M. and Annie Zaenen. 1989. Long-distance dependencies, constituent structure, and functional uncertainty. In Mark Baltin and Anthony Kroch (eds.), *Alternative conceptions of phrase structure*, 17–42. Chicago: University of Chicago Press.
- Kuhn, Jonas. 2003. *Optimality-theoretic syntax: A declarative approach*. Stanford, CA: CSLI Publications.

Prince, Alan and Paul Smolensky. 2004. *Optimality theory: Constraint interaction in generative grammar*. Malden, MA: Blackwell. Originally circulated in 1993 as Technical Report 2, Rutgers University Center for Cognitive Science.

Spencer, Piyapath. forthcoming. Category and composition from position: Evidence from thai auxiliaries .

A Requirements and Setup

A.1 Requirements

- XLE (Crouch et al., 2011). Obtained separately, under PARC’s terms.
- Tcl (interpreter embedded in XLE; nothing to install to run `xlerc` or `parse`)
- Aqua Tcl/Tk 8.6 (macOS GUI only). `tcltk/`, `tcl8.6/` and `tk8.6-aqua/` must already sit beside the XLE binary; `setup.sh` copies them into the bundle, it does not supply them (§ A.3).
- Python 3 on PATH as `python3` or `python`.
- `pythainlp`: `pip install pythainlp`.

A.2 Files

File	Description
<code>thai.lfg</code>	main (config) file, FILES-includes the rest
<code>thai-rules.lfg</code>	RULES (§ 2.1)
<code>thai-templates.lfg</code>	TEMPLATES
<code>thai-features.lfg</code>	FEATURES: the TAME geometry
<code>thai-lex.lfg</code>	LEXICON: every word the fragment knows
<code>testsuite.lfg</code>	45 sentences: 35 grammatical, 10 ungrammatical
<code>common.features.lfg</code>	shared (STANDARD COMMON) features
<code>common.templates.lfg</code>	shared templates (SUBJ, OBJ, SUBJ-SUBJ-CONTROL, ...)
<code>xlerc</code>	XLE runtime config; instatiates the tokeniser, loads the grammar
<code>utils/th-tok.py</code>	Thai segmenter (§ A.4)
<code>utils/setup.sh</code>	macOS environment setup (§ A.3)
<code>utils/setup.ps1</code>	Windows equivalent

A.3 Setup: macOS

XLE’s Tk interface otherwise requires X11 via XQuartz. Run `utils/setup.sh` once, from the directory holding the XLE binary and the `tcltk/`, `tcl8.6/`, `tk8.6-aqua/` runtimes. It:

- wraps the binary in an `XLE.app` bundle, Aqua Tcl/Tk copied inside;
- writes a launcher setting `XLEPATH`, `DYLD_LIBRARY_PATH`, `TCL_LIBRARY`, `TK_LIBRARY` to the bundled copies;
- ad-hoc code-signs the bundle;
- appends the same variables to `~/.zshrc`.

A.4 Tokenisation

Thai is written without word spaces. XLE’s tokeniser splits on whitespace, so raw Thai arrives as one unknown token and nothing parses.

`xlerc` renames XLE’s `parse` to `_xle_parse` and defines a wrapper:

- input contains U+0E00–U+0E7F → pass to `utils/th-tok.py`, echo the segmentation, hand the space-separated result to `_xle_parse`;
- otherwise → `_xle_parse` unchanged.

`th-tok.py` uses `pythainlp`’s dictionary tokeniser, with a dictionary built from `thai-lex.lfg` rather than `pythainlp`’s general Thai dictionary. Consequences:

- adding a word to the lexicon adds it to the tokeniser;
- the tokeniser recognises only words this fragment knows.

Both files locate their neighbours relative to themselves, so the folder can be moved. `testsuite.lfg` is stored unsegmented, so the suite runs through `parse`: every count in § 4 exercises tokeniser and grammar together. `_xle_parse` isolates the two.

B Gloss of All Examples

Every sentence in `testsuite.lfg`, glossed in full. Numbering matches `testsuite.lfg` exactly. Grammatical sentences note the expected solution count (§ 4); every ungrammatical sentence expects exactly 0.

B.1 Grammatical Examples

- (1) Bare SV

เขา ทำ
k^hǎw t^ham
3SG do
‘(S)he does it.’

- (2) SVO

เขา กิน ข้าว
k^hǎw kin k^hâ:w
3SG eat rice
‘(S)he eats rice.’

- (3) Intransitive (§ 3.6)

เขา เดิน
k^hǎw dɤ:n
3SG walk
‘(S)he walks.’

- (4) SVC - 2 verbs (§ 3.6)
 เขา เดิน กิน ข้าว
 k^hǎw dɔːn kin k^hâ:w
 3SG walk eat rice
 ‘(S)he walks over and eats rice.’
- (5) SVC - 4 verbs (§ 3.6)
 เขา เดิน ออก ไป กิน ข้าว
 k^hǎw dɔːn ʔò:k paj kin k^hâ:w
 3SG walk exit go eat rice
 ‘(S)he walks out to eat rice.’
- (6) SVC - 5 verbs (§ 3.6)
 เขา เดิน ออก ไป นั่ง กิน ข้าว
 k^hǎw dɔːn ʔò:k paj nâŋ kin k^hâ:w
 3SG walk exit go sit eat rice
 ‘(S)he walks out, sits down, and eats rice.’
- (7) SVC + object control
 เขา เดิน ออก ไป บอก แม่ ให้ กิน ข้าว
 k^hǎw dɔːn ʔò:k paj bò:k mĕː hâj kin k^hâ:w
 3SG walk exit go tell mother COMP eat rice
 ‘(S)he walks out and tells mom to eat rice.’
- (8) Object dropped, null pro supplied by OBJ-SUBJ-CONTROL (§ 3.6)
 เขา บอก ให้ กิน ข้าว
 k^hǎw bò:k hâj kin k^hâ:w
 3SG tell COMP eat rice
 ‘(S)he tells someone to eat.’
- (9) evaluative mood
 เขา คง ทำ
 k^hǎw k^hoŋ t^ham
 3SG MOOD_{eval} do
 ‘(S)he’s probably doing it.’
- (10) Realis aspect, pre-verbal
 เขา ไปได้ ทำ
 k^hǎw dâj t^ham
 3SG MOOD_{real} do
 ‘(S)he did it.’
- (11) Ambiguous: post-verbal ไปได้ (Completive/Ability heterosemy) 2 Solutions (§ 5)
 เขา ทำ ไปได้
 k^hǎw t^ham dâj
 3SG do ASP_{abil}

‘(S)he can do it.’

- (12) Perfective aspect

เขา ทำ แล้ว
k^hǎw t^ham lé:w
3SG do ASP_{pftv}

‘(S)he’s already done it.’

- (13) Ambiguous: post-verbal ได้ (Completive/Ability ambiguity) + perfective

2 Solutions

เขา ทำ ได้ แล้ว
k^hǎw t^ham dâj lé:w
3SG do ASP_{abil} ASP_{pftv}

‘(S)he can already do it.’

- (14) Perfective + 2 clause-final particles (Speech Act + Hedge)

เขา ทำ แล้ว ละ มั้ง
k^hǎw t^ham lé:w lâ? máŋ
3SG do ASP_{pftv} MOOD_{sfp} MOOD_{hedge}

‘(S)he’s probably done it already, I’d guess.’

- (15) Weak alethic necessity (§ 3.1)

เขา ควร ทำ
k^hǎw k^hu:an t^ham
3SG MOD_{alet.wk} do

‘(S)he should do it.’

- (16) progressive aspect

เขา กำลัง ทำ
k^hǎw kam.laŋ t^ham
3SG ASP_{prog} do

‘(S)he’s doing it.’

- (17) Realis aspect + transitive object

เขา ได้ กิน ข้าว
k^hǎw dâj kin k^hâ:w
3SG MOOD_{real} eat rice

‘(S)he got to eat rice.’

- (18) Transitive object + post-verbal ได้ ambiguity

2 Solutions

เขา กิน ข้าว ได้
k^hǎw kin k^hâ:w dâj
3SG eat rice ASP_{abil}

‘(S)he can eat rice.’

- (19) Weak alethic > Strong alethic > progressive

เขา ควร ต้อง กำลัง ทำ
 k^hǎw k^hu:an tɔ̃:ŋ kam.laŋ t^ham
 3SG MOD_{alet.wk} MOD_{alet.str} ASP_{prog} do
 ‘(S)he really ought to be doing it by now.’

- (20) Weak alethic > progressive > Deontic ต้อง (different order from above still grammatical)

เขา ควร กำลัง ต้อง ทำ
 k^hǎw k^hu:an kam.laŋ tɔ̃:ŋ t^ham
 3SG MOD_{alet.wk} ASP_{prog} MOD_{obl} do
 ‘(S)he should be having to do it right now.’

- (21) เคย as past then anterior around the irrealis modal จะ (§ 3.3)

เขา เคย จะ เคย ทำ
 k^hǎw k^hɤ:j tɕàʔ k^hɤ:j t^ham
 3SG TNS_{pst} MOOD_{irr} TNS_{ant} do
 ‘(S)he’d apparently done it before.’

- (22) The full stack, but with 2 post-verbal ได้ 2 Solutions

เขา คง อาจ จะ ควร ต้อง กำลัง จะ ได้
 k^hǎw k^hoŋ ʔà:t tɕàʔ k^hu:an tɔ̃:ŋ kam.laŋ tɕàʔ dâj
 3SG MOOD_{eval} MOD_{epi} MOOD_{irr} MOD_{alet.wk} MOD_{alet.str} ASP_{prog} ASP_{prosp} MOOD_{real}
 ทำ ได้ แล้ว ละ มั้ง
 t^ham dâj lé:w lâʔ máŋ
 do ASP_{abil} ASP_{pftv} MOOD_{sfp} MOOD_{hedge}
 ‘Perhaps (s)he is probably supposed to be able to do it by now.’

- (23) Weak alethic + Transitive object

เขา ควร กิน ข้าว
 k^hǎw k^hu:an kin k^hâ:w
 3SG MOD_{alet.wk} eat rice
 ‘(S)he should eat rice.’

- (24) progressive กำลัง + Transitive object

เขา กำลัง กิน ข้าว
 k^hǎw kam.laŋ kin k^hâ:w
 3SG ASP_{prog} eat rice
 ‘(S)he’s eating rice.’

- (25) Weak alethic + OBJ + Post-verbal ได้ ambiguity

2 Solutions

เขา ควร กิน ข้าว ได้
 k^hǎw k^hu:an kin k^hâ:w dâj
 3SG MOD_{alet.wk} eat rice ASP_{abil}
 ‘(S)he should be able to eat rice.’

- (26) evaluative + OBJ + Perfective

เขา คง กิน ข้าว แล้ว
 kʰǎw kʰoŋ kin kʰâ:w lé:w
 3SG MOOD_{eval} eat rice ASP_{pftv}
 ‘(S)he’s probably already eaten rice.’

- (27) ต้อง’s strong-alethic and deontic readings are both structurally licensed here 2 Solutions (§ 5)

เขา ต้อง กิน ข้าว
 kʰǎw tɔ̃:ŋ kin kʰâ:w
 3SG MOD_{alet.str}/MOD_{obl} eat rice
 ‘(S)he has to eat rice.’

- (28) ควร (Weak alethic) directly followed by ต้อง before an OBJ-taking verb 2 Solutions

เขา ควร ต้อง กิน ข้าว
 kʰǎw kʰu:an tɔ̃:ŋ kin kʰâ:w
 3SG MOD_{alet.wk} MOD_{alet.str}/MOD_{obl} eat rice
 ‘(S)he really ought to eat rice.’

- (29) SVC with modal (§ 3.6)

เขา ควร เดิน กิน ข้าว
 kʰǎw kʰu:an dɔ̃:n kin kʰâ:w
 3SG MOD_{alet.wk} walk eat rice
 ‘(S)he should walk over and eat rice.’

- (30) SVC with auxiliary

เขา กำลัง เดิน ไป กิน ข้าว
 kʰǎw kam.laŋ dɔ̃:n paj kin kʰâ:w
 3SG ASP_{prog} walk go eat rice
 ‘(S)he’s walking over to eat rice.’

- (31) SVC with post-verbal auxiliary

เขา เดิน กิน ข้าว แล้ว
 kʰǎw dɔ̃:n kin kʰâ:w lé:w
 3SG walk eat rice ASP_{pftv}
 ‘(S)he already walked over and ate rice.’

- (32) SVC with modal ambiguity as #27/#28

2 Solutions

เขา ควร ต้อง เดิน ไป กิน ข้าว
 kʰǎw kʰu:an tɔ̃:ŋ dɔ̃:n paj kin kʰâ:w
 3SG MOD_{alet.wk} MOD_{alet.str}/MOD_{obl} walk go eat rice
 ‘(S)he really needs to go eat rice.’

- (33) SVC + modal + object control

เขา คง เดิน ออก ไป บอก แม่ ให้ กิน ข้าว
 kʰǎw kʰoŋ dɔ̃:n ?ò:k paj bò:k mɛ̃: hâj kin kʰâ:w
 3SG MOOD_{eval} walk exit go tell mother COMP eat rice
 ‘(S)he’s probably walked out to tell mom to eat.’

- (34) SVC + pre- and post-verbal auxiliaries

เขา ได้ เดิน ออก ไป กิน ข้าว แล้ว
 k^hǎw dâj dɤ:n ʔò:k paj kin k^hâ:w lé:w
 3SG MOOD_{real} walk exit go eat rice ASP_{pftv}
 ‘(S)he already went out and ate.’

- (35) The sentence #22’s entire pre- and post-verbal stack governing the SVC + object-control construction instead of a bare verb (§ 3.6)

เขา คง อาจ จะ ควร ต้อง กำลัง จะ ได้
 k^hǎw k^hoŋ ʔà:t tɕàʔ k^hu:an tɔ̃:ŋ kam.laŋ tɕàʔ dâj
 3SG MOOD_{eval} MOD_{epi} MOOD_{irr} MOD_{alet.wk} MOD_{alet.str} ASP_{prog} ASP_{prosp} MOOD_{real}
 เดิน ออก ไป บอก แม่ ให้ กิน ข้าว ได้ แล้ว ละ มั่ง
 dɤ:n ʔò:k paj bò:k mɛ: hâj kin k^hâ:w dâj lé:w lâʔ máŋ
 walk exit go tell mother COMP eat rice ASP_{abil} ASP_{pftv} MOOD_{sfp} MOOD_{hedge}
 ‘Perhaps (s)he’s probably supposed to have already gone and told mom to eat — and apparently can, too.’

B.2 Ungrammatical Examples

- (36) epistemic cannot precede evaluative (positions 2, 1)

*เขา อาจ คง ทำ
 k^hǎw ʔà:t k^hoŋ t^ham
 3SG MOD_{epi} MOOD_{eval} do
 intended: ‘perhaps (s)he probably does it’

- (37) Strong alethic cannot precede weak alethic (positions 7, 6)

*เขา ต้อง ควร ทำ
 k^hǎw tɔ̃:ŋ k^hu:an t^ham
 3SG MOD_{alet.str} MOD_{alet.wk} do
 intended: ‘(s)he must be supposed to do it’

- (38) progressive cannot precede weak alethic (§ 3.5)

*เขา กำลัง ควร ทำ
 k^hǎw kam.laŋ k^hu:an t^ham
 3SG ASP_{prog} MOD_{alet.wk} do
 intended: ‘(s)he’s currently supposed to do it’

- (39) Perfective cannot precede postverbal ability (positions 13, 12)

*เขา ทำ แล้ว ได้
 k^hǎw t^ham lé:w dâj
 3SG do ASP_{pftv} ASP_{abil}
 intended: ‘(s)he’s already done it, and can’

- (40) Realis aspect is a single flat slot, occupied at most once

*เขา ได้ ได้ กิน ข้าว
 k^hǎw dâj dâj kin k^hâ:w
 3SG MOOD_{real} MOOD_{real} eat rice

intended: ‘(s)he already-already ate rice’

- (41) Realis cannot precede evaluative (positions 11, 1)

*เขา ใต้ คง ทำ
k^hǎw dâj k^hoŋ t^ham
3SG MOOD_{real} MOOD_{eval} do
intended: ‘(s)he did probably do it’

- (42) Same violation as sentence 39, with an object present

*เขา กิน ข้าว แล้ว ใต้
k^hǎw kin k^hâ:w lɛ:w dâj
3SG eat rice ASP_{pftv} ASP_{abil}
intended: ‘(s)he’s already eaten rice, and can’

- (43) Same violation as sentence 38, with an object present

*เขา กำลัง ควร กิน ข้าว
k^hǎw kam.laŋ k^hu:an kin k^hâ:w
3SG ASP_{prog} MOD_{alet.wk} eat rice
intended: ‘(s)he’s currently supposed to eat rice’

- (44) Same violation as sentence 36, with an object present

*เขา อาจ คง กิน ข้าว
k^hǎw ʔà:t k^hoŋ kin k^hâ:w
3SG MOD_{epi} MOOD_{eval} eat rice
intended: ‘perhaps (s)he probably eats rice’

- (45) Same violation as sentence 38/43, with an SVC chain present

*เขา กำลัง ควร เดิน กิน ข้าว
k^hǎw kam.laŋ k^hu:an dɔ:n kin k^hâ:w
3SG ASP_{prog} MOD_{alet.wk} walk eat rice
intended: ‘(s)he’s currently supposed to walk over and eat rice’